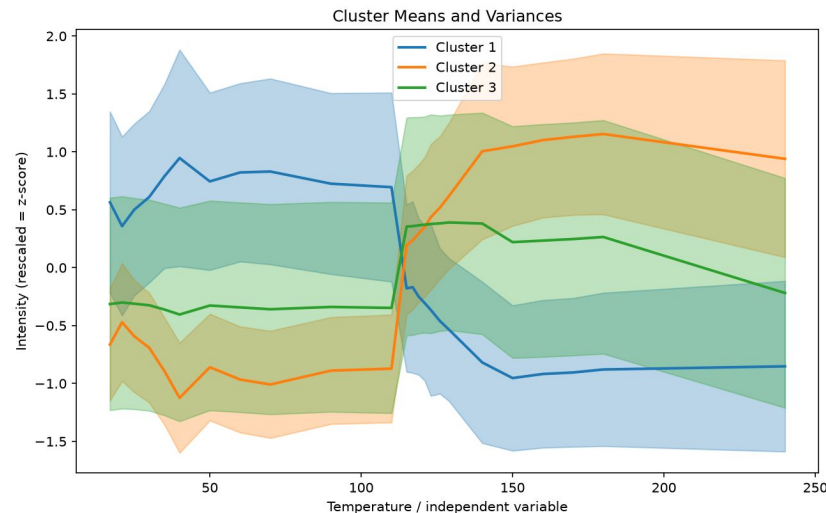
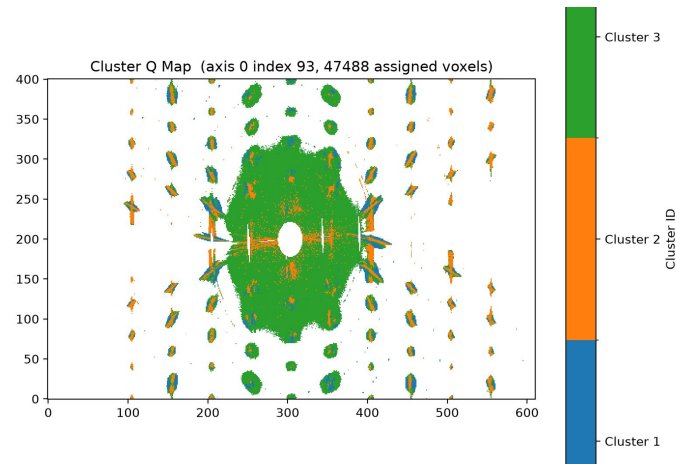


Building a Helper

The Problem

- On z-score the judge lands on $k=3$, but the winner changes across rescale methods
- Rescale changes the answer more than any scoring weight does
- Scores aren't comparable across rescales
- The weights were hand picked for different data



Weight Findings

- Only one weight actually matters
 - Swept each weight 0->5; five of six never change
 - Only w_{bic} can flip it
 - $k=3$ wins on every term except BIC
- Two terms are not contributing
 - $size_penalty = 0.000$ for every candidate
 - $jump_penalty$ penalizes everyone equally, so it doesn't separate any
- Really a two term judge
 - Minimal model (separation + coherence) reproduces the judge's $k=3$

Metric Findings

BIC Elbow

Why: BIC's lowest value picks too many clusters; experts read where the curve flattens

Found: seemed promising when I ran it on my data, tested it on synthetic data and it picked $k=3$ no matter what the real answer was

Junk cluster

Why: bad clusters are flat and noisy

Found: Worked on mean and z-score

Trajectory Redundancy

Why: we don't want two clusters that look alike, so it rewards clusterings where trajectories are unique

Found: every cluster shares the same jump at ~110K, so the metric saw them all as similar and flagged everything

Metric Findings

Amplitude

Why: good trajectories rise or fall clearly. Rewarded clusterings whose trajectories moved the most

Found: it was able to tell good scaling methods from bad ones, but it didn't pick the right number of clusters

Q-space compactness

Why: real clusters are tight, bright spots in reciprocal space. Scored how tightly each cluster's points group together in space.

Found: it always favored fewer clusters, since fewer always looks tighter

Using Bradley-Terry Model to learn the weights instead of guessing

- Right now the judges' weights are hand picked
 - We're guessing at what matters and by how much
- Once we have metrics that actually capture a good clustering, we could use Bradley-Terry to learn the weights from expert choices instead of guessing and checking
- How it works: you show it which clustering an expert preferred, and it finds the weights that reproduce that choice
- What is needed: a handful of expert labeled datasets (ideally a ranking of options)